Mohammad Idrees Bhat

Tech Skills Trainer | AI/ML Consultant

Data Cleaning with MySQL

--

Contents:

- Data Cleaning
- Handling Missing Values
- Handling Duplicates
- Data Type Conversions
- Extra Topics
  - Pivot in MySQL
  - Conditional Aggregation
  - Unpiviot
  - Views - How are they different

Mohammad Idrees Bhat

Invent yourself, and then reinvent, yourself.

What is a Data Cleaning?

Data cleaning is the process of finding, correcting, removing, or handling inaccurate, incomplete, inconsistent, or irrelevant data so that the dataset becomes reliable and suitable for analysis.

In simple words:

> Data cleaning means preparing messy data so that it can be trusted and used for analysis.

- Real-world data is rarely perfect.
- Data can come from:
  - Excel files
  - Databases
  - APIs
  - Websites
  - Surveys
  - Customer forms
  - Sensors
  - Multiple systems

Because of this, data may contain mistakes such as:

Problem	Example
Missing values	`salary = NULL`
Duplicate records	Same employee appears again and again
Inconsistent spelling	`Kashmir`, `kashmir`, `KASHMIR`
Inconsistent Categories	US, USA, United States
Wrong data type	`salary = "50 thousand"`
Invalid values	`age = 250`
Incorrect dates	`2024-99-45`

Problem	Example
Extra spaces	`" Srinagar "`
Inconsistent formats	`India`, `IND`, `Indian`
Data type issues Problems	Text type date formats

> If we perform analysis on dirty data, our results can be misleading

Let's begin learning.

But we will also need data for the implementation.

Here is some data simulated to teach the topics.

In [ ]:
```
USE store_management; -- mention the database you have in your MySQL

DROP TABLE IF EXISTS employees_raw;

CREATE TABLE employees_raw (
    emp_id INT,
    name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10, 2),
    country VARCHAR(50),
    join_date VARCHAR(20)
);

INSERT INTO employees_raw (emp_id, name, department, salary, country, join_date) VALUES
(1, 'Alice', 'Sales', 60000, 'US', '2023-01-15'), (2, 'Bob', 'Sales', 65000, 'US', '2023-02-10'),
(3, 'Charlie', 'Sales', NULL, 'US', '2023-03-05'), (4, 'Diana', 'IT', 80000, 'US', '2023-04-20'),
(5, 'Evan', 'IT', 75000, 'UK', '2023-05-12'), (6, 'Fiona', 'HR', 55000, 'US', '2023-06-18'),
(7, 'George', 'HR', 60000, 'US', '2023-07-22'), (8, 'Derek', 'Sales', 65000, 'CA', '2023-08-30'),
(9, 'Elena', 'Sales', NULL, 'US', '2023-09-14'), (10, 'Felix', 'Sales', 50000, 'US', '2023-10-01'),
(4, 'Diana', 'IT', 80000, 'US', '2023-04-20'), -- Duplicate
(11, 'Grace', 'Marketing', 70000, 'US', '2023-11-05'), (12, 'Henry', 'Marketing', NULL, 'US', '2023-11-20'),
(13, 'Ivy', 'IT', 82000, 'US', '2023-12-01'), (14, 'Jack', 'HR', 58000, 'AU', '2024-01-10'),
(15, 'Kelly', 'Sales', 62000, 'US', '2024-02-15'), (16, 'Liam', 'IT', 78000, 'US', '2024-03-22'),
```

```
(17, 'Mia', 'Marketing', 71000, 'US', '2024-04-18'), (18, 'Noah', 'Sales', 64000, 'US', '2024-05-05'),
(19, 'Olivia', 'HR', NULL, 'US', '2024-06-12'), (20, 'Paul', 'IT', 85000, 'US', '2024-07-25'),
(15, 'Kelly', 'Sales', 62000, 'US', '2024-02-15'), -- Duplicate
(21, 'Quinn', 'Marketing', 69000, 'UK', '2024-08-30'), (22, 'Ryan', 'Sales', 61000, 'US', '2024-09-14'),
(23, 'Sophia', 'HR', 57000, 'US', '2024-10-02'), (24, 'Tom', 'IT', NULL, 'US', '2024-11-11'),
(25, 'Uma', 'Marketing', 72000, 'US', '2024-12-05'), (26, 'Victor', 'Sales', 66000, 'US', '2025-01-20'),
(27, 'Wendy', 'IT', 79000, 'CA', '2025-02-15'), (28, 'Xavier', 'HR', 59000, 'US', '2025-03-10'),
(29, 'Yara', 'Sales', 63000, 'US', '2025-04-22'), (30, 'Zane', 'Marketing', 73000, 'US', '2025-05-18'),
(31, 'Alice_B', 'Sales', 60500, 'US', '2025-06-01'), (32, 'Bob_B', 'IT', 76000, 'US', '2025-06-15'),
(22, 'Ryan', 'Sales', 61000, 'US', '2024-09-14'), -- Duplicate
(33, 'Charlie_B', 'HR', NULL, 'US', '2025-07-10'), (34, 'Diana_B', 'Marketing', 74000, 'US', '2025-07-25'),
(35, 'Evan_B', 'Sales', 67000, 'US', '2025-08-05'), (36, 'Fiona_B', 'IT', 81000, 'UK', '2025-08-20'),
(37, 'George_B', 'HR', 61000, 'US', '2025-09-10'), (38, 'Derek_B', 'Marketing', 68000, 'US', '2025-09-28'),
(39, 'Elena_B', 'Sales', NULL, 'US', '2025-10-15'), (40, 'Felix_B', 'IT', 83000, 'US', '2025-11-02'),
(41, 'Grace_B', 'HR', 62000, 'US', '2025-11-20'), (42, 'Henry_B', 'Marketing', 75000, 'US', '2025-12-10'),
(43, 'Ivy_B', 'Sales', 68000, 'US', '2026-01-05'), (44, 'Jack_B', 'IT', 84000, 'CA', '2026-01-22'),
(45, 'Kelly_B', 'HR', NULL, 'US', '2026-02-15'), (46, 'Liam_B', 'Marketing', 76000, 'US', '2026-03-01'),
(34, 'Diana_B', 'Marketing', 74000, 'US', '2025-07-25'), -- Duplicate
(47, 'Mia_B', 'Sales', 69000, 'US', '2026-03-20'), (48, 'Noah_B', 'IT', 86000, 'US', '2026-04-10'),
(49, 'Olivia_B', 'HR', 63000, 'US', '2026-04-28'), (50, 'Paul_B', 'Marketing', 77000, 'US', '2026-05-15'),
(51, 'Quinn_B', 'Sales', 70000, 'US', '2026-06-05'), (52, 'Ryan_B', 'IT', NULL, 'US', '2026-06-25'),
(53, 'Sophia_B', 'HR', 64000, 'UK', '2026-07-12'), (54, 'Tom_B', 'Marketing', 78000, 'US', '2026-07-30'),
(55, 'Uma_B', 'Sales', 71000, 'US', '2026-08-18'), (56, 'Victor_B', 'IT', 87000, 'US', '2026-09-05'),
(57, 'Wendy_B', 'HR', 65000, 'US', '2026-09-25'), (58, 'Xavier_B', 'Marketing', NULL, 'US', '2026-10-15'),
(48, 'Noah_B', 'IT', 86000, 'US', '2026-04-10'), -- Duplicate
(59, 'Yara_B', 'Sales', 72000, 'US', '2026-11-02'), (60, 'Zane_B', 'IT', 88000, 'US', '2026-11-20'),
(61, 'Alice_C', 'HR', 66000, 'CA', '2026-12-10'), (62, 'Bob_C', 'Marketing', 79000, 'US', '2027-01-05'),
(63, 'Charlie_C', 'Sales', 73000, 'US', '2027-01-25'), (64, 'Diana_C', 'IT', 89000, 'US', '2027-02-15'),
(65, 'Evan_C', 'HR', NULL, 'US', '2027-03-08'), (66, 'Fiona_C', 'Marketing', 80000, 'US', '2027-03-28'),
(67, 'George_C', 'Sales', 74000, 'US', '2027-04-15'), (68, 'Derek_C', 'IT', 90000, 'UK', '2027-05-05'),
(69, 'Elena_C', 'HR', 67000, 'US', '2027-05-25'), (70, 'Felix_C', 'Marketing', 81000, 'US', '2027-06-15'),
(64, 'Diana_C', 'IT', 89000, 'US', '2027-02-15'), -- Duplicate
(71, 'Grace_C', 'Sales', NULL, 'US', '2027-07-02'), (72, 'Henry_C', 'IT', 91000, 'US', '2027-07-22'),
(73, 'Ivy_C', 'HR', 68000, 'US', '2027-08-10'), (74, 'Jack_C', 'Marketing', 82000, 'CA', '2027-08-30'),
(75, 'Kelly_C', 'Sales', 75000, 'US', '2027-09-18'), (76, 'Liam_C', 'IT', 92000, 'US', '2027-10-08'),
(77, 'Mia_C', 'HR', 69000, 'US', '2027-10-28'), (78, 'Noah_C', 'Marketing', NULL, 'US', '2027-11-15'),
(79, 'Olivia_C', 'Sales', 76000, 'UK', '2027-12-05'), (80, 'Paul_C', 'IT', 93000, 'US', '2027-12-25'),
(72, 'Henry_C', 'IT', 91000, 'US', '2027-07-22'), -- Duplicate
(81, 'Quinn_C', 'HR', 70000, 'US', '2028-01-12'), (82, 'Ryan_C', 'Marketing', 83000, 'US', '2028-02-02'),
(83, 'Sophia_C', 'Sales', 77000, 'US', '2028-02-22'), (84, 'Tom_C', 'IT', 94000, 'US', '2028-03-15'),
(85, 'Uma_C', 'HR', NULL, 'US', '2028-04-05'), (86, 'Victor_C', 'Marketing', 84000, 'CA', '2028-04-25'),
(87, 'Wendy_C', 'Sales', 78000, 'US', '2028-05-15'), (88, 'Xavier_C', 'IT', 95000, 'US', '2028-06-05'),
```

```
(89, 'Yara_C', 'HR', 71000, 'US', '2028-06-25'), (90, 'Zane_C', 'Marketing', 85000, 'US', '2028-07-15'),
(84, 'Tom_C', 'IT', 94000, 'US', '2028-03-15'), -- Duplicate
(91, 'Alice_D', 'Sales', 79000, 'UK', '2028-08-02'), (92, 'Bob_D', 'IT', NULL, 'US', '2028-08-22'),
(93, 'Charlie_D', 'HR', 72000, 'US', '2028-09-10'), (94, 'Diana_D', 'Marketing', 86000, 'US', '2028-09-30'),
(95, 'Evan_D', 'Sales', 80000, 'US', '2028-10-20'), (96, 'Fiona_D', 'IT', 96000, 'US', '2028-11-10'),
(97, 'George_D', 'HR', 73000, 'CA', '2028-11-30'), (98, 'Derek_D', 'Marketing', 87000, 'US', '2028-12-20'),
(99, 'Elena_D', 'Sales', 81000, 'US', '2029-01-10'), (100, 'Felix_D', 'IT', 97000, 'US', '2029-01-30'),
(95, 'Evan_D', 'Sales', 80000, 'US', '2028-10-20'), -- Duplicate
(101, 'Grace_D', 'HR', NULL, 'US', '2029-02-18'), (102, 'Henry_D', 'Marketing', 88000, 'US', '2029-03-10'),
(103, 'Ivy_D', 'Sales', 82000, 'US', '2029-03-30'), (104, 'Jack_D', 'IT', 98000, 'UK', '2029-04-20'),
(105, 'Kelly_D', 'HR', 74000, 'US', '2029-05-10'), (106, 'Liam_D', 'Marketing', 89000, 'US', '2029-05-30'),
(107, 'Mia_D', 'Sales', 83000, 'US', '2029-06-20'), (108, 'Noah_D', 'IT', 99000, 'US', '2029-07-10'),
(109, 'Olivia_D', 'HR', 75000, 'US', '2029-07-30'), (110, 'Paul_D', 'Marketing', NULL, 'CA', '2029-08-20'),
(108, 'Noah_D', 'IT', 99000, 'US', '2029-07-10'), -- Duplicate
(111, 'Quinn_D', 'Sales', 84000, 'US', '2029-09-08'), (112, 'Ryan_D', 'IT', 100000, 'US', '2029-09-28'),
(113, 'Sophia_D', 'HR', 76000, 'US', '2029-10-18'), (114, 'Tom_D', 'Marketing', 90000, 'US', '2029-11-05'),
(115, 'Uma_D', 'Sales', 85000, 'UK', '2029-11-25'), (116, 'Victor_D', 'IT', 101000, 'US', '2029-12-15'),
(117, 'Wendy_D', 'HR', NULL, 'US', '2030-01-05'), (118, 'Xavier_D', 'Marketing', 91000, 'US', '2030-01-25'),
(119, 'Yara_D', 'Sales', 86000, 'US', '2030-02-15'), (120, 'Zane_D', 'IT', 102000, 'CA', '2030-03-05'),
(118, 'Xavier_D', 'Marketing', 91000, 'US', '2030-01-25'), -- Duplicate
(121, 'Alice_E', 'HR', 77000, 'US', '2030-03-25'), (122, 'Bob_E', 'Marketing', 92000, 'US', '2030-04-15'),
(123, 'Charlie_E', 'Sales', 87000, 'US', '2030-05-05'), (124, 'Diana_E', 'IT', 103000, 'US', '2030-05-25'),
(125, 'Evan_E', 'HR', 78000, 'US', '2030-06-15'), (126, 'Fiona_E', 'Marketing', NULL, 'UK', '2030-07-05'),
(127, 'George_E', 'Sales', 88000, 'US', '2030-07-25'), (128, 'Derek_E', 'IT', 104000, 'US', '2030-08-15'),
(129, 'Elena_E', 'HR', 79000, 'US', '2030-09-05'), (130, 'Felix_E', 'Marketing', 93000, 'CA', '2030-09-25'),
(128, 'Derek_E', 'IT', 104000, 'US', '2030-08-15'), -- Duplicate
(131, 'Grace_E', 'Sales', 89000, 'US', '2030-10-15'), (132, 'Henry_E', 'IT', 105000, 'US', '2030-11-05'),
(133, 'Ivy_E', 'HR', 80000, 'US', '2030-11-25'), (134, 'Jack_E', 'Marketing', 94000, 'US', '2030-12-15'),
(135, 'Kelly_E', 'Sales', NULL, 'US', '2031-01-05'), (136, 'Liam_E', 'IT', 106000, 'US', '2031-01-25'),
(137, 'Mia_E', 'HR', 81000, 'UK', '2031-02-15'), (138, 'Noah_E', 'Marketing', 95000, 'US', '2031-03-05'),
(139, 'Olivia_E', 'Sales', 90000, 'US', '2031-03-25'), (140, 'Paul_E', 'IT', 107000, 'US', '2031-04-15'),
(138, 'Noah_E', 'Marketing', 95000, 'US', '2031-03-05'), -- Duplicate
(141, 'Quinn_E', 'HR', 82000, 'CA', '2031-05-05'), (142, 'Ryan_E', 'Marketing', 96000, 'US', '2031-05-25'),
(143, 'Sophia_E', 'Sales', 91000, 'US', '2031-06-15'), (144, 'Tom_E', 'IT', 108000, 'US', '2031-07-05'),
(145, 'Uma_E', 'HR', 83000, 'US', '2031-07-25'), (146, 'Victor_E', 'Marketing', 97000, 'US', '2031-08-15'),
(147, 'Wendy_E', 'Sales', 92000, 'US', '2031-09-05'), (148, 'Xavier_E', 'IT', NULL, 'US', '2031-09-25'),
(149, 'Yara_E', 'HR', 84000, 'UK', '2031-10-15'), (150, 'Zane_E', 'Marketing', 98000, 'US', '2031-11-05');
```

- Simple method for **Removing Irrelevant Data**
  - Imagine our current analysis is strictly for the US offices. We need to filter out the noise.

In [ ]:
```
-- The <> symbol means "Not Equal To"
DELETE FROM employees_raw
WHERE country <> 'US';
```

Handling Null Values

First Rule of Data Cleaning -> Don't Immediately Change the Data

One of the biggest mistakes beginners make is:

> "I found NULL values. Let me update them."

First, Profile the data. Identify the mess properly.

- understand what is wrong before making any modifications

Some questions one can start with:

- how many records do we have?
- how many unique employees/customers
- are there any null values?
- which columns are most compromised?

In [ ]:
```
-- use these commands
SELECT *
FROM employees_raw;

SELECT COUNT(*) AS total_rows
FROM employees_raw;

DESCRIBE employees_raw;
-- notice that join_date is stored as a VARCHAR(20)
-- already a quality issue
```

Check NULL values in columns

In [ ]:
```sql
-- The IS NULL operator is how SQL checks for emptiness
SELECT COUNT(*) AS missing_salaries
FROM employees_raw
WHERE salary IS NULL;
-- GROUP BY department; -- if data too big

-- oberve those records
SELECT *
FROM employees_raw
WHERE salary IS NULL;
```

Handling NULL Values

- We use two functions generally to solve this

1. `IFNULL()`

- Takes exactly two arguments: the column you want to check and the backup value to use if that column is NULL
- The issue is:
  - `IFNULL()` is specific to MySQL and SQLite.
  - If they ever get a job using PostgreSQL or SQL Server, this function won't work! That is why we usually prefer the next option.

In [ ]:
```sql
SELECT
    name,
    department,
    salary AS actual_salary,
    -- If salary is NULL, replace it with $45,000
    IFNULL(salary, 45000.00) AS plan_b_salary
FROM employees_raw
WHERE salary IS NULL; -- Filtering just to show the affected rows
```

2. `COALESCE()`

- Looks at a list and picks the first thing that isn't NULL
- It is more powerful and is universal standard across all SQL languages

In [ ]:
```sql
SELECT
    name,
    department,
    COALESCE(salary, 45000.00) AS fixed_salary
FROM employees_raw;
```

> Note: IFNULL(salary, 45000) does the exact same thing in MySQL, but COALESCE is the universal standard across all SQL languages

True Power of COALESCE

- Why COALESCE is superior, give them a hypothetical scenario where
- Say an employee has multiple contact numbers

In [ ]:
```sql
-- An example to show the class how COALESCE keeps searching
SELECT
    name,
    COALESCE(cell_phone, home_phone, work_phone, 'No Contact Provided') AS best_contact_number
FROM hypothetical_contacts;
```

- If `cell_phone` is null, it checks `home_phone`
- If `home_phone` is null, it checks `work_phone`
- If they are all null, it outputs the final text string

3. Handling Missing Data - using Window Functions

- Using window functions we can make data imputations
- In data cleaning and data science, the process of replacing missing (NULL) values with substituted values is called Data Imputation (or simply Imputation)
- Imputation means replacing missing (NULL) values with a reasonable value instead of leaving them missing.
- Some times one has to make use of existing knowlege or domain expertise to choose the correct type of value to fill

In [ ]:
```sql
SELECT
    name,
    department,
    salary AS original_salary,
    -- If salary is null, grab the average salary in this department
    COALESCE(
        salary,
        AVG(salary) OVER(PARTITION BY department ORDER BY emp_id)
    ) AS imputed_salary
FROM employees_raw;
```

Common imputation strategies:

Strategy	Window Function
Previous value - Forward Fill	`LAG()`
Next value - Backward Fill	`LEAD()`
Group average	`AVG() OVER()`
Group sum	`SUM() OVER()`
Group minimum	`MIN() OVER()`
Group maximum	`MAX() OVER()`
Group median	`PERCENTILE_CONT()` - does not work in mySQL

In [ ]:
```sql
-- For LAG -  AKA Forward Fill

SELECT employee_id, name, salary,
       LAG(salary) OVER (ORDER BY employee_id) AS prev_salary
FROM employees;
```

In [ ]:
```sql
-- For LEAD - AKA Backward Fill
SELECT employee_id, name, salary,
       LEAD(salary) OVER (ORDER BY employee_id)
FROM employees;
```

In [ ]:
```sql
-- For Group average
SELECT employee_id, name, salary,
       AVG(salary) OVER (PARTITION BY department) -- use SUM or Min or MAX
FROM employees;
```

In [ ]:
```sql
-- For Group average
-- works in Pgsql, oracle, sql server, snowflake, amazon redshift
SELECT employee_id, name, salary,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) OVER (PARTITION BY department)
FROM employees;
```

Making Updates to the table

- How does this work?

1. Calculate the imputation values in a subquery or a CTE, and then use the UPDATE function
   - The CTE creates an invisible results table named imputed that exists just for the duration of this query
   - The window function calculates the average and attaches it to every single employee row
   - COALESCE looks at the list of values and returns the first one that is not NULL
     - Checks salary and returns it
     - If salary is NULL it takes the average value
2. Then the updation needs to be done
   - We use Inner Join (in SQL just writing JOIN defaults to INNER JOIN)
   - JOIN directly inside an UPDATE is a dialect specific feature
     - i.e. it works mostly in MySQL and MariaDB
   - It JOINs the `employees_raw` table with the fresly created CTE table `imputed`
   - `SET` takes newly calculated `imputed_salary` column and overwrites the blank `salary` column
   - WHERE clause acts as the safety net and performance boost
     - only affect the rows where salary is NULL

In [ ]:
```sql
--
WITH imputed AS (
       SELECT
       emp_id,
       COALESCE(
           salary,
```

```
            AVG(salary) OVER(PARTITION BY department)
        ) AS imputed_salary
    FROM employees_raw
)
UPDATE employees_raw e
JOIN imputed i
    ON e.emp_id = i.emp_id
SET e.salary = i.imputed_salary
WHERE e.salary IS NULL;
```

In [ ]:
```
-- In postgres, sqlite, redshift, snowflake, bigquery
WITH imputed AS (
    SELECT
        emp_id,
        COALESCE(
            salary,
            AVG(salary) OVER(PARTITION BY department)
        ) AS imputed_salary
    FROM employees_raw
)
UPDATE employees_raw e
SET salary = i.imputed_salary
FROM imputed i
WHERE e.emp_id = i.emp_id        -- This replaces the "ON" clause from the JOIN
  AND e.salary IS NULL;          -- This is the safety filter
```

Handling Duplicates

Check Duplicates values

- If a company accidentally runs a payroll script twice, employees might be inserted twice
- We can find this by grouping the exact same data together and seeing if the count is greater than 1.

In [ ]:
```
-- Scenario: We want a list of unique employee names and departments.
-- DISTINCT forces SQL to hide any row that is an exact 100% match to another row.

SELECT DISTINCT
```

```
    emp_id, name, department
FROM employees_raw;
```

In [ ]:
```
-- another way to just observe the duplicates
SELECT
    emp_id, name, department,
    COUNT(*) as frequency
FROM employees_raw
GROUP BY emp_id, name, department
HAVING COUNT(*) > 1;
```

"DISTINCT is like wearing sunglasses that filter out duplicates. The duplicates are still in the database; you just aren't looking at them right now.

- We can use this keyword directly to remove duplicates as well
-

In [ ]:
```
-- to remove duplicates
CREATE TABLE employees_clean AS
SELECT DISTINCT * FROM employees_raw;
```

Then additionally, one could do this:

- Thus, we would have a clean dataset without any duplicates

In [ ]:
```
DROP TABLE employees_raw;

-- Rename the clean table to the original name
ALTER TABLE employees_clean RENAME TO employees_raw;
```

- We could also just only view every data row just once
- Using GROUP BY on all the columns, we group any duplicates together
- The duplicates still remain in the data, but our Selection does not contain any duplicates
- Just like this:

In [ ]:
```
SELECT
    emp_id, name, department, salary, country, join_date
```

```
FROM employees_raw
GROUP BY
    emp_id, name, department, salary, country, join_date;
```

In [ ]:
```
-- can clear duplicates like this too

-- Step 1: Create a staging table using our GROUP BY query
CREATE TABLE employees_stage AS
SELECT
    emp_id, name, department, salary, country, join_date
FROM employees_raw
GROUP BY
    emp_id, name, department, salary, country, join_date;

-- Step 2: Swap the tables
DROP TABLE employees_raw;
RENAME TABLE employees_stage TO employees_raw;
```

- We could use a Ranking function to check all the duplicate values now
- Industry standard way is to use a CTE for this - **for SQL Server**
- ROW_NUMBER() inside a Common Table Expression (CTE) to attach a unique number to every row.
- This is a good opportunity to learn about CTEs

What is a CTE? - (OPTIONAL to learn)

A **Common Table Expression** (CTE) is just a temporary, named result set. CTEs simplify complex queries, make them easier to read and can be reused multiple times within the same query

Think of it as a sticky note. You write a query, stick it on the wall with a name, and then you can query that sticky note directly.

Here is a basic starter example:

In [ ]:
```
WITH IT_Department AS (
    SELECT * FROM employees_raw WHERE department = 'IT'
)
```

```sql
-- Now we query the "sticky note" we just created
SELECT name, salary FROM IT_Department WHERE salary > 80000;
```

In [ ]:
```sql
WITH AvgSalaryByDept AS (
    SELECT Department, AVG(Salary) AS AvgSalary
    FROM employees_raw
    GROUP BY Department
)
SELECT *
FROM AvgSalaryByDept;
```

Some databases limit using CTEs with INSERT, UPDATE or DELETE.

Using a CTE to Delete Duplicates

- The command `SELECT DISTINCT * FROM employees_raw` will show you a clean list, but it doesn't delete the bad rows from the table. To actually delete them, we use a CTE and ROW_NUMBER().
- We will number every row. If we see a duplicate emp_id, the first one gets a 1, and the duplicate gets a 2. Then, we simply delete anything greater than 1

In [ ]:
```sql
-- Scenario: We want to permanently delete the clone rows while keeping the originals.

WITH DuplicateFinder AS (
    SELECT
        emp_id,
        name,
        department,
        -- The window function assigns a 1 to the first row it sees,
        -- a 2 to the first clone, a 3 to the second clone, etc.
        -- Assign a 1, 2, 3 based on identical emp_ids
        ROW_NUMBER() OVER (
            PARTITION BY emp_id, name, department
            ORDER BY emp_id -- We just need an order, the column doesn't matter much here
        ) AS row_num
    FROM employees_raw
)

-- Now we can safely delete anything that isn't a "1"
```

```
DELETE FROM DuplicateFinder
WHERE row_num > 1;
```

- But there is an issue with this method.
- It works fine in MS SQL Server
- But MySQL or PostgreSQL **does not allow direct deletion** from a CTE
- The reason is that they treat CTEs as strictly **Read-only**

In [ ]:
```
-- so the way is to create a new clean table
CREATE TABLE employees_clean AS
WITH DuplicateFinder AS (
    SELECT
        emp_id, name, department, salary, country, join_date,
        ROW_NUMBER() OVER (
            PARTITION BY emp_id, name, department
            ORDER BY emp_id
        ) AS row_num
    FROM employees_raw
)
-- Select only the columns you want to keep (leaving row_num behind)
SELECT
    emp_id, name, department, salary, country, join_date
FROM DuplicateFinder
WHERE row_num = 1;
```

Data Type Conversions

- Sometimes data is clean, but it's wearing the wrong clothes. Look at our join_date.
- When we created the table, we saved it as a VARCHAR (Text).
- If we try to do date math (like finding out how many days someone has worked here), SQL will crash because it thinks the date is just a word.

How do we check the type of data columns?

Almost all major relational databases (SQL Server, PostgreSQL, MySQL) support the `INFORMATION_SCHEMA`. This is a system database that stores metadata about your tables.

In [ ]:
```
SELECT
    column_name,
    data_type
FROM
    information_schema.columns
WHERE
    table_name = 'employees_raw';



-- Other methods:
DESCRIBE employees_raw;

-- Check all tables
SHOW COLUMNS FROM your_table_name;
```

- The most common format issues appear in the date columns.
- You fix the type issues using the CAST() Function
- CAST() is an ANSI SQL standard function.
  - This means it works exactly the same way across almost every major relational database (MySQL, PostgreSQL, SQL Server, Oracle, SQLite).

```
CAST(expression AS target_data_type)
```

1. `CAST()`

- The Universal Standard
- `CAST is the cleanest, most standard way to change data types.`
- If the date is stored as text (a `VARCHAR` or `STRING`), but it is already perfectly formatted like '2026-03-15' (Standard YYYY-MM-DD), you can use the CAST function. This simply tells the database, "Treat this text as a date."

In [ ]:
```
SELECT
    name,
    join_date AS text_date,
```

```
    CAST(join_date AS DATE) AS real_date
FROM employees_raw;
```

2. `CONVERT()`

- The Formatter
- Powerful because it allows a third argument, style code
- Syntax can vary between databases
- CONVERT works in SQL Server, allows you to reformat the output completely using a style code (e.g. 101 -> mm/dd/yyyy)
- Other codes are:
  - 101 'MM/DD/YYYY'
  - 103 'DD/MM/YYYY'
  - 104 'DD.MM.YYYY'
  - 120 'YYYY-MM-DD'

```
-- In SQL Server, these make sense
CONVERT(target_data_type, expression, [style_code])


-- Converts a Date object into a US-styled string
SELECT CONVERT(VARCHAR(10), GETDATE(), 101);
-- NOTE: Changes the stle of an already created Date Object, not a text column

-- You cannot use style codes with CONVERT() in MySQL or PgSQL
```

In MySQL Standard type conversion looks like this:

- `CONVERT(expression, target_data_type)`
  - e.g. `CONVERT('2026-08-11', DATE)`
- Notice the arguments are flipped compared to SQL Server `SELECT CONVERT(DATE, join_date);`

In [ ]:
```
-- Try these in MySQL
SELECT
    name,
```

```
    CONVERT(join_date, DATE) AS proper_date
FROM employees_raw;
```

In [ ]:
```
-- To change stlye use the DATE_FORMAT() function
SELECT
    name,
    join_date,
    -- Formats the date as mm/dd/yyyy
    DATE_FORMAT(join_date, '%m/%d/%Y') AS format_date
FROM employees_raw;
```

- NOTE: `%d` gets day, `%m` extracts month and `%y` gets year
- One can try any text compositions using this
- `'%d/%y-%m'` - should work fine too
  - And so will `'%d+%y-idrees-%m'`

Feature	`CAST()`	`CONVERT()`
Purpose	Converts a value from one data type to another	Converts a value from one data type to another
Syntax	`CAST(value AS data_type)`	`CONVERT(value, data_type)` Different Syntax in SQL Server
Readability	More standardized and generally easier to read	MySQL-specific syntax is also straightforward
SQL Standard	ANSI SQL standard	MySQL's `CONVERT(expr, type)` syntax is not ANSI-standard
Data types supported	Common types such as `CHAR`, `DATE`, `DATETIME`, `DECIMAL`, `SIGNED`, `UNSIGNED`, etc.	Similar type conversions
Portability	Better if moving SQL between database systems	More specific to MySQL and SQL Server (uses style codes too)
When to prefer	General-purpose data type conversion	MySQL-specific conversions, especially character sets

- **To make these changes permanent in database**

In [ ]:
```
ALTER TABLE employees_raw
MODIFY COLUMN join_date DATE;
```

In [ ]:
```
-- Or just use the UPDATE with the formatted strings using this:
-- DATE_FORMAT(join_date, '%m+%d+%Y');

-- But before that, you will need to make sure that the Join_date is string type - so
-- use this first
ALTER TABLE employees_raw
MODIFY COLUMN join_date VARCHAR(20);

-- Then apply the custom format
UPDATE employees_raw
SET join_date = DATE_FORMAT(join_date, '%m+%d+%Y');
```

In [ ]:
```
-- NOTE: Might need to do this, for this to work

-- 1. Turn off the safety net
SET SQL_SAFE_UPDATES = 0;

-- 2. Run your update
UPDATE employees_raw
SET join_date = DATE_FORMAT(join_date, '%m+%d+%Y');

-- 3. Turn the safety net back on (Best Practice)
SET SQL_SAFE_UPDATES = 1;
```

Extra Topics

PIVOT Table

- A pivot table is a data summarization tool that transforms a long, flat list of raw data into a compact, multi-dimensional grid to reveal patterns and calculate totals instantly.

- It allows you to flip (or "pivot") columns into rows and rows into columns to see the data from different angles without writing complex formulas.

A pivot table performs three actions simultaneously:

1. **Rotates data:** It takes unique values from a single column (like country) and flips them to become their own individual columns across the top.
2. **Groups data:** It establishes a base category for your rows (like department).
3. **Aggregates data:** It calculates the math (SUM, COUNT, AVG) where the rows and columns intersect.

- Understand the concept:
- Example:
  - If you have 1,000 rows of individual employee records
  - Pivot table crushes that down into a clean 4x4 summary grid
  - Showing Total Salary by Department (rows) and Country (columns).

- Unlike SQL Server or Snowflake, MySQL does not have a native **PIVOT** keyword
- To pivot data in MySQL, we use a technique called **Conditional Aggregation**

CONDITIONAL AGGREGATION

- CASE statement is SQL's version of IF-THEN-ELSE logic
-
- It allows you to look at your data, check if it meets a specific condition, and return a specific value based on that condition

In [ ]:
```
SELECT
    name,
    department,
    CASE
        WHEN department = 'Sales' THEN 'Yes!'
        ELSE 'No'
```

```
    END AS Salesss
FROM employees_raw;
```

In [ ]:
```
-- example on employees table
SELECT
    name,
    salary,
    CASE
        WHEN salary >= 75000 THEN 'High'
        WHEN salary >= 60000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_bracket
FROM employees;
```

Now let's use this to PIVOT our data

In [ ]:
```
-- Take one category, US employees and use that to Pivot data
SELECT
    department,
    -- If the country is US, count the employee. Otherwise, ignore them
    COUNT(CASE WHEN country = 'US' THEN emp_id END) AS US_Employees,
     -- We can also add a total column easily
    COUNT(emp_id) AS Total_Employees
FROM employees_raw
GROUP BY department;
```

Working:

- The `GROUP BY department` instruction tells MySQL to create groups - one row for IT, one for Sales, one for HR, etc
- As MySQL looks at the 'Sales' row, it scans every Sales employee
- The `CASE WHEN country = 'US'` statement checks the country
  - If it's a match, it hands the `emp_id` to the `COUNT()` function
  - If it's not a match, it implicitly returns `NULL`, which `COUNT()` ignores
- Do the same thing for other categories and you have the table

In [ ]:
```
-- Pivot table for mulitple countries
-- CREATE TABLE department_headcount AS -- To save the table, uncomment
```

```
SELECT
    department,
    -- If the country is US, count the employee. Otherwise, ignore them.
    COUNT(CASE WHEN country = 'US' THEN emp_id END) AS US_Employees,
    COUNT(CASE WHEN country = 'UK' THEN emp_id END) AS UK_Employees,
    COUNT(CASE WHEN country = 'CA' THEN emp_id END) AS CA_Employees,
    COUNT(CASE WHEN country = 'AU' THEN emp_id END) AS AU_Employees,

    -- We can also add a total column easily
    COUNT(emp_id) AS Total_Employees
FROM employees_raw
GROUP BY department;
```

- NOTE: Add a `CREATE TABLE table_name AS` on top to save the pivot table

UNPIVOT

- Think of unpivoting as taking a pair of scissors to your wide table
- You cut off each column, manually label it, and stack them vertically on top of one another
- Let's assume you saved our previous pivot table into a new table called `department_headcount`
    - currently has columns for department, US_Employees, UK_Employees, CA_Employees, and AU_Employees

In [ ]:
```
-- Query 1: Slice out the US data
SELECT
    department,
    'US' AS country,          -- Hardcoding the category name
    US_Employees AS employee_count
FROM department_headcount

UNION ALL

-- Query 2: Slice out the UK data, stack it underneath
SELECT
    department,
    'UK' AS country,
    UK_Employees AS employee_count
```

```
FROM department_headcount

UNION ALL

-- Query 3: Slice out the CA data, stack it underneath
SELECT
    department,
    'CA' AS country,
    CA_Employees AS employee_count
FROM department_headcount

UNION ALL

-- Query 4: Slice out the AU data, stack it underneath
SELECT
    department,
    'AU' AS country,
    AU_Employees AS employee_count
FROM department_headcount;
```

Views

- A View is a saved SQL query that behaves like a table
- Instead of storing a separate copy of the data, a view stores the SQL query used to retrieve the data

> A View is a virtual table created from a SQL query.

In [ ]:
```
CREATE VIEW high_salary_employees AS
SELECT
    emp_id,
    name,
    department,
    salary
FROM employees_raw
WHERE salary > 50000;
```

- You build your table or pivot table let's say, and save it as a view

- then connect your dashboards directly to it for real time metrics without exposing your messy raw data

Views are the industry standard for **Data Security** and **Privacy**

- In the real world, your employees_raw table contains highly sensitive information
- Specifically, the salary column.
- You cannot give the IT helpdesk or a company directory application direct access to that table,
- but they still need a live list of employee names, IDs, and departments.

In [ ]:
```
CREATE VIEW vw_public_directory AS
SELECT
    emp_id,
    name,
    department,
    country
    -- Notice we intentionally exclude the salary column
FROM employees_raw
WHERE salary IS NOT NULL; -- Only show completed records
```

How are VIEWS different?

Feature	Physical Table	View	CTE (`WITH ... AS`)
What is it?	Physical storage of data.	A saved query behaving like a virtual table.	A temporary, named result set.
Disk Space?	Yes. It physically holds data.	Minimal. Only the query logic is saved.	None. Exists only during query execution.
Data Freshness	Static. Requires manual updates or triggers.	Live. Pulls fresh data the moment it is queried.	Live. Evaluated precisely when the query runs.
Lifespan	Permanent (until explicitly dropped).	Permanent (until explicitly dropped).	Execution only. Dies instantly when the query finishes.
Best Use Case	Archiving historical data or storing heavy datasets that are too slow to calculate live.	Hiding complex logic, restricting user access to specific columns, or BI reporting.	Breaking down massive, multi-step calculations into readable chunks inside a single script.

Summary

Live Exercise

END

THANK YOU!

Live Exercise Solutions

Task 1 solution

```
# Starter code for Exercise 1 (students can modify)

query_ex1 = """
SELECT
    day,
    AVG(size) AS avg_size
FROM orders
GROUP BY day
ORDER BY avg_size DESC;
"""

avg_size_by_day = pd.read_sql_query(query_ex1, conn)
avg_size_by_day
```

In [ ]:

```
# Starter code for Exercise 2

query_ex2 = """
SELECT
    smoker,
    AVG(tip_pct) AS avg_tip_pct
FROM orders
GROUP BY smoker;
"""

tip_pct_by_smoker = pd.read_sql_query(query_ex2, conn)
tip_pct_by_smoker
```

Task 2 solution

In [ ]:

Mohammad Idrees Bhat

Tech Skills Trainer | AI/ML Consultant